LLM Classification Project

Ian Rocklein

Problem Definition:
The goal of this project is to create CNN and RNN (LSTM) models and see if we can classify which LLM wrote a certain response. This will be accomplished by gathering the data, training the model and fine-tuning the parameters for the model, then testing multiple situations such as input only, output only, and input + output and comparing how the classifiers perform and how accurately they are able to make these classifications.

Data Curation:
Data Source: lmsys/chatbot_arena_conversations · Datasets at Hugging Face

- The user prompts are licensed under CC-BY-4.0, while the model outputs are licensed under CC-BY-NC-4.0.

I got the data for this project from a Hugging Face dataset. The original dataset includes two models that use the same prompt, and then a user decided which was better. For the creation of the dataset for this project, I only considered the model_a column to prevent too many prompt duplicates or anything of that sort. There was also a language aspect where some examples used a prompt and response in a different language, so I also filtered it out so there were only English prompts and responses. The data included different types of prompts, including some math and coding questions outside of the classic chat prompts. I simplified the data to only include claude-v1, gpt-4, and koala-13b. I did this by writing a Python script that accessed the dataset through Hugging Face's datasets library and parsed through it, adding tuples to a text file in the form (LLM_name, LLM_input, LLM_output).

The data ended up having about 950 points for each LLM, but after analyzing the dataset, there were some duplicates within it, which were cleared out by removing duplicates from the dataframe in the main code section. So the data is not perfectly balanced, but there are some discrepancies.

CNN/RNN Implementation

Data and Preprocessing

As previously mentioned, the dataset comes from the lmsys/chatbot_arena_conversations collection and contains responses from three LLMs: koala-13b, gpt-4, and claude-v1. Each example consists of a model label, the user prompt, and the model's response. After loading, I repaired surrogate-pair encoding errors (caused by emojis), dropped rows with missing values, and removed duplicate prompt/response pairs. Labels were encoded as integers with scikit-learn's LabelEncoder.

Splitting. I split on unique prompts rather than on individual rows, using 70% train, 15% validation, and 15% test (random_state=42). Because each prompt appears once per model, this keeps all responses to the same prompt in the same split. That prevents leakage, where the

model could see a prompt's responses in training and be tested on the same prompt, and it means the models must generalize to unseen prompts.

Tokenization. I used spaCy's blank English tokenizer. I preserve case and punctuation, since certain styles like capitalization, punctuation habits, and formatting may be what distinguishes the models. Newlines are kept as a special <NL> token so paragraph and list structure remains visible and removed other whitespace.

Vocabulary and sequences: The vocabulary is built from the training set only. It includes every token appearing at least 5 times, plus <pad> (index 0) and <unk> (index 1). Out-of-vocabulary tokens map to <unk>. Sequences are truncated or zero-padded to a fixed length equal to the 95th percentile of training response lengths ([max_len] tokens), which avoids letting a few very long responses dominate memory and padding. True lengths (minimum 1) are stored for use by the LSTM.

Convolutional Neural Network (CNN):
I chose to design this model as a multi-kernel CNN. It also includes a learned embedding layer with 100 dimensions, a padding index of 0, and a dropout of 0.2. The small dropout on the embeddings can help prevent overfitting to specific tokens, which matters with a fairly small dataset.

I also used parallel 1D convolution branches that each have 100 filters, and are over multiple kernel sizes. Each branch applies a 1D convolution with 100 filters, followed by batch normalization and ReLU, and the kernel sizes are in the range of 2-4 or 2-5 which ends up getting chosen during tuning. Using several kernel widths lets the model detect patterns of different lengths at once, batch normalization stabilizes and speeds up training, and ReLU adds the nonlinearity that lets filters respond only to strong matches.

It also includes global max-pooling over time for each filter, keeping the strongest match anywhere in the response, and the pooled outputs of all branches are concatenated. Since patterns can appear anywhere in the text, max-pooling makes the detection independent of position, and collapses the variable-length responses into a fixed-size feature vector.

The pooled features pass through dropout and then a linear layer producing one score for each LLM class. The concatenated feature vector is large in comparison to the amount of training data, so by using a heavier dropout here, it is able to build more of a defense so the model does not memorize training responses.

Recurrent Neural Network (RNN):
The design for the RNN was done as an LSTM to completely avoid any issues from the vanishing gradient problem. As far as the embeddings go, it uses the exact same settings as the CNN, with 100 dimensions, padding index 0, and an embedding dropout of 0.2.

I chose to make it a bidirectional LSTM to better recognize how each LLM usually opens their response and how they close it. It is a single-layer LSTM that uses a base of 128 for its hidden dimension, but in the tuning process we also tested 64. Unlike the CNN, which only looks for local n-gram patterns, an LSTM carries information across the whole sequence, so it can pick up on longer-range structure such as how a response is organized.

Before the LSTM, responses are packed with their true lengths using pack_padded_sequence, so the padding is skipped. Since the responses got padded to be a fixed length, if we didn't pack the zeros the LSTM and the state would reflect the padding instead of the actual response.

The last forward and last backward hidden states are then concatenated into a single vector of size 2 × hidden_dim. The final state of the LSTM acts as a summary of everything it has read, so taking the final state from each direction gives a fixed-size representation of the whole response regardless of its length.

The combined hidden state passes through dropout and a linear layer, producing one score per LLM. There are many parameters relative to this dataset and can cause this model to overfit easily, so including a strong dropout before the output layer acts as regularization. Also ended up including weight decay that was tuned in training as an additional regularizer.

Training Process:
Both models use the same training setup, this makes sure any differences in results come from the model and not the training process. Also to ensure it was reproducible I used a fixed random seed before every run (random_seed(42))

For the *loss function,* I used cross-entropy, it is pretty standard for a multi-class classification, and it directly penalizes the model for putting low probability on the correct LLM.

For the optimizer, I used Adam because it adapts the step size for each parameter which is helpful for inputs like text. I set the learning rate to 0.001 and it is consistent for both models in order to keep comparisons fair.

I used a batch size of 32 and shuffled the training set. Using a smaller batch size gives weight updates more often, and introduces more noise that can help regularize training. Shuffling the epoch was done to prevent the models from learning a particular order of examples.

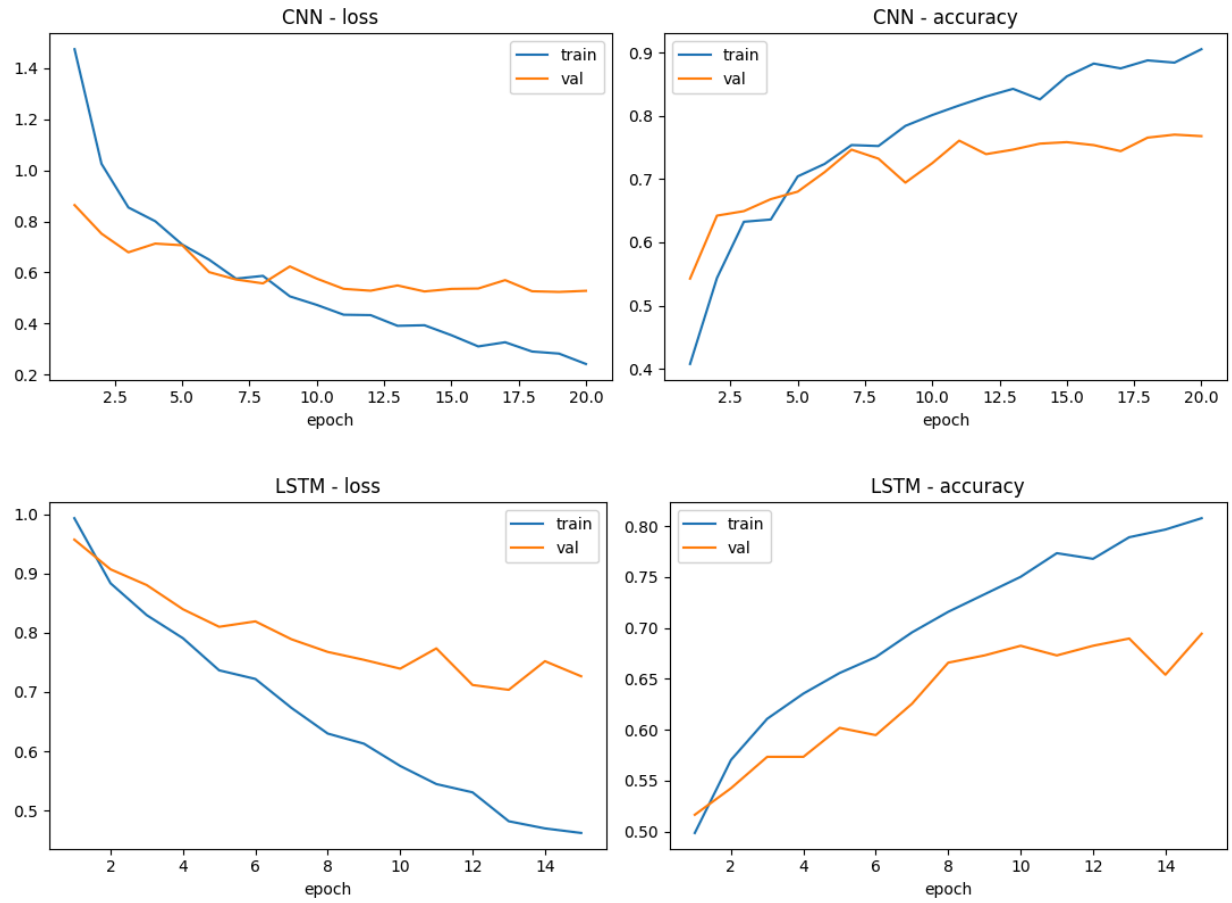
I ended up doing 20 epochs for the CNN and 15 epochs for the LSTM model. This was decided on through some trial and error, based on when a model would start overfitting, while also making sure it had plenty of room to reach its ceiling. The LSTM initially would start overfitting quickly after around 4 epochs, but once I started introducing weight decay and more fine-tuning of the parameters, more epochs were able to be introduced.

After each epoch, the model is evaluated on the validation set, and the weights from the epoch with the lowest validation loss are restored at the end. Training loss keeps improving as the

model memorizes the training data, but validation loss shows when generalization stops improving. By restoring the best epoch, we are able to give the same protection as early stopping without needing to guess the right stopping point in advance.

For regularization during training, the CNN uses dropout, embedding dropout, batch normalization, and an optional learning-rate scheduler. The LSTM uses dropout, embedding dropout, and optional weight decay. In the hyperparameter tuning, I used a nested for loop to iterate through multiple different parameter options I thought were reasonable. The CNN went through 12 different configurations, covering dropout {0.3, 0.5, 0.6}, kernel sizes {(2,3,4), (2,3,4,5)}, and scheduler {off, on}. The LSTM went through 12 configurations as well, with dropout {0.5, 0.6}, hidden size {64, 128}, and weight decay {0, 1e-3, 1e-4}.

Finally, the best configuration for each model is evaluated once on the held-out test set, which was split by prompt so no prompt appears in more than one split. I report accuracy, macro-F1, a per-class classification report, and a confusion matrix.



Research Questions

RQ1: Can we identify which LLM generated a response?

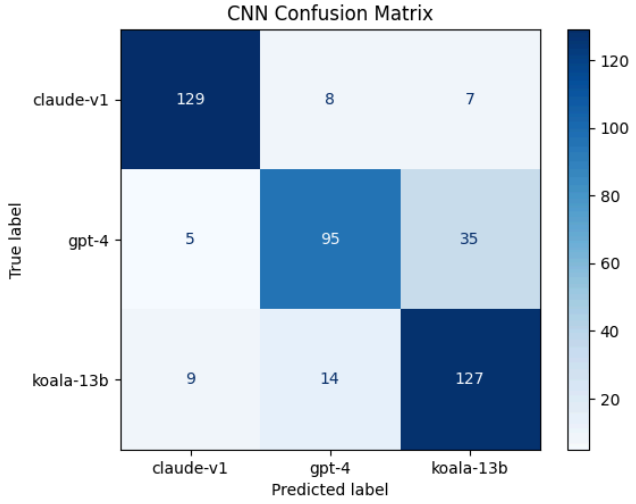
- Use LLM_output as input, and conduct classification to detect which LLM generated it

CNN Results:

CNN classification report

Test set, 429 responses | accuracy 0.818 | macro-F1 0.816

	precision	recall	f1-score	support
claude-v1	0.90	0.90	0.90	144
gpt-4	0.81	0.70	0.75	135
koala-13b	0.75	0.85	0.80	150
accuracy			*0.82*	*429*
macro avg	*0.82*	*0.82*	*0.82*	*429*
weighted avg	*0.82*	*0.82*	*0.82*	*429*



The CNN model was very successful in identifying the Claude-v1 responses. The 0.90 precision rate means that the model was correctly labeling the outputs as Claude 90% of the time, and the 0.90 recall rate means that the model was able to find and identify 90% of the true Claude outputs. This is noticeably stronger than what was found for both GPT-4 and Koala-13b. GPT-4 was the worst of all, with an f1-score of 0.75, mostly coming from the weaker recall score of 0.70. Even though it had a decent precision score, 30% of true GPT-4 outputs were mislabeled. While the Koala-13b data had a solid recall score of 0.85, it lacked precision, only 75% of the outputs the model labeled Koala-13b were actually Koala-13b.

Looking at the confusion matrix, it is visible that the CNN model struggled most when it labeled GPT-4 outputs as Koala-13b outputs. It labeled about 26% of the true GPT-4 outputs as

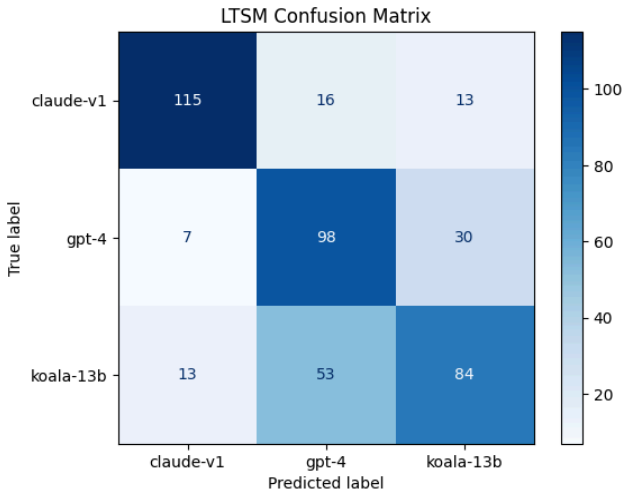
Koala-13b, so it was often getting confused between the two. But what is interesting is that it was one-way, while the model did not struggle nearly as much in accidentally labeling Koala outputs as GPT-4 outputs. This asymmetry could suggest that the model leans toward predicting Koala-13b when GPT-4 outputs lack distinctive n-grams, such as in cases of short or generic responses.

LSTM Results:

LSTM classification report

Test set, 429 responses | accuracy 0.692 | macro-F1 0.693

	precision	recall	f1-score	support
claude-v1	0.85	0.80	0.82	144
gpt-4	0.59	0.73	0.65	135
koala-13b	0.66	0.56	0.61	150
accuracy			*0.69*	*429*
macro avg	*0.70*	*0.69*	*0.69*	*429*
weighted avg	*0.70*	*0.69*	*0.69*	*429*



The LSTM model was the most successful in identifying Claude-v1, with an overall f1-score of 0.82. It struggled more with both Koala-13b and GPT-4. Koala-13b did the worst with an overall f1-score of 0.61. It did substantially worse when it came to recall, only being able to identify just over half of its own true labeled outputs. When the model did label an output as Koala-13b, it was moderately accurate, being correct 66% of the time. The model had a slightly better f1-score of 0.65 for GPT-4, with a strong recall score and a lackluster precision score, opposite of how the model performed on Koala-13b. The recall score was 0.73, compared to a precision score of 0.59.

Looking at the confusion matrix, this model was very confused between GPT-4 and Koala-13b in both directions, classifying 22.2% of true GPT-4 outputs as Koala-13b, and 35.3% of true Koala-13b outputs classified as GPT-4.

Comparison:
The CNN outperformed the LSTM overall, with a test accuracy of 0.818 and macro-F1 of 0.816 versus 0.692 and 0.693 for the LSTM, and it had higher f1-scores on every class, with Claude-v1 0.90 vs. 0.82, Koala-13b 0.80 vs. 0.61, GPT-4 0.75 vs. 0.65. Both models ranked Claude-v1 best and struggled the most to distinguish GPT-4 from Koala-13b, which suggests that Claude's outputs have the most distinct style. Though with the CNN's errors were mostly one-directional with 25.9% of true GPT-4 outputs labeled Koala-13b, but only 9.3% of Koala-13b labeled GPT-4. On the other hand the LSTM was confused both ways, mislabeling Koala-13b as GPT-4 in 35.3% of cases, which is why its Koala recall (0.56) and GPT-4 precision (0.59) were so low. Overall, these results suggest that the CNN's local n-gram patterns were a more reliable authorship signal than the LSTM's sequence-level representation, most likely because of the small dataset and because it is more difficult to compress long responses into a single hidden state.

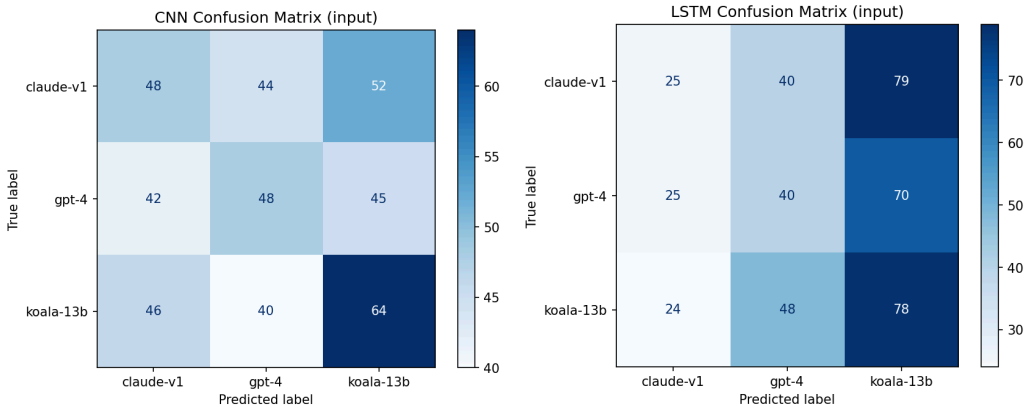
RQ2: Does the user's prompt help identify the LLM? Compare classifiers using: Input only; Output only; Input + Output

To set up this experiment, I tokenized the inputs in the same way I tokenized the output to do RQ1. For the input + output, each prompt and its response were tokenized separately with the same spaCy tokenizer, then joined into one sequence with a <SEP> token between them. Then for both the LSTM and CNN models, I used the best parameters that were found in RQ1 to set up the models with. For the CNN this was a dropout of 0.3, kernel sizes of (2, 3, 4) and using the scheduler. For the LSTM, this included a dropout of 0.6, a hidden dimension of 64, and a weight decay of 0.001.

I also iterated through 3 different seeds and found the average statistics across them because certain seeds have certain variations, and the different seeds might have slightly different outcomes so I wanted to create a larger sample size. These were seeds 42, 43, 44. Then based on these three seeds, I calculated the mean and standard deviation for the test accuracy and macro_f1 scores. All the confusion matrices in this section come from seed 44, so the output confusion matrices might look slightly different than what was shown in RQ1. There will be a brief description of what the confusion matrices look like for the other seeds, but they will not all be included in this report to keep things clearer. The other confusion matrices are also included in the Jupyter notebook if you choose to reference them.

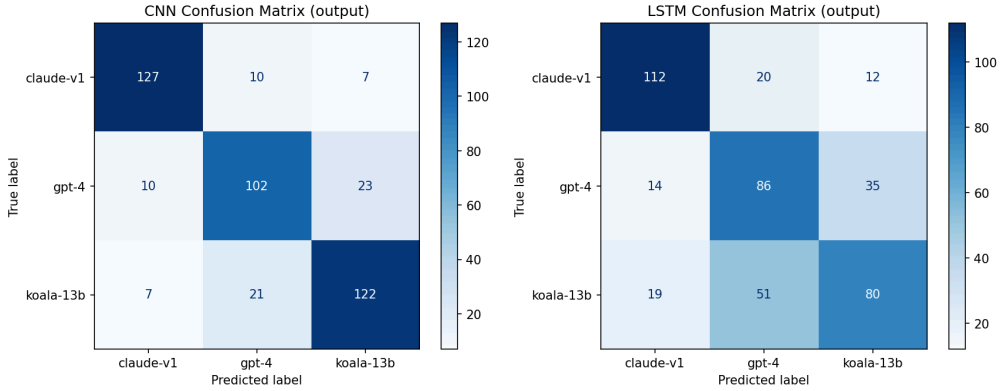
condition	model	test_acc		macro_f1	
		mean	std	mean	std
both	CNN	0.793	0.028	0.792	0.027
	LSTM	0.574	0.045	0.575	0.047
input	CNN	0.377	0.007	0.375	0.009
	LSTM	0.331	0.004	0.322	0.005
output	CNN	0.821	0.013	0.819	0.012
	LSTM	0.658	0.030	0.659	0.030

Input Only:



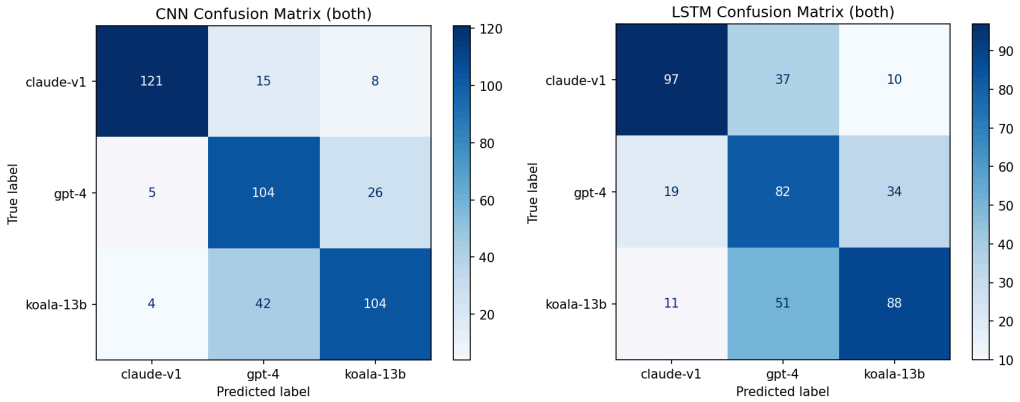
The LSTM matrix may look like there is some pattern, but seed 42 was much more evenly distributed, and seed 43 basically flipped Koala and Claude's columns. CNN was pretty evenly distributed for all seeds. Averaged over the 3 seeds, the CNN had a mean test accuracy of 0.377, and the LSTM had a mean of 0.331. Since random guessing among three balanced classes gives 0.333, the LSTM performed exactly at chance. Its macro-f1 of 0.322 is slightly lower than its accuracy because its predictions were concentrated on one class in some runs, which drags down the f1 of the under-predicted classes. Though the CNN model was slightly above that rate, I wouldn't say there is enough evidence to state that it could meaningfully predict which prompt was put into which model. Which is what we would expect since the prompt is written before any model responds. Though it could be possible in a much larger dataset curated across many people; if say GPT-4 has much better image generation or Claude is much better when it comes to coding or math, maybe the input only would have higher separation. In the context of this dataset, where it is only in the 900s for each class, there is not a wide enough variety of prompts that you might find in a 50,000-prompt dataset.

Output Only:



Both of these confusion matrices follow very similar pattern to what was already described in RQ1, where Claude-v1 gets identified pretty consistently, but the model struggles a little bit more in deciding between GPT-4 and Koala-13b. The confusion matrices for the other seeds follow pretty much the same pattern too. The mean macro-f1 for the CNN model was 0.819, which is consistent with what was found for RQ1. The LSTM had a mean macro-f1 of 0.659, which was slightly less than the 0.693 found in RQ1 but still close to the range of the 0.030 standard deviation. We also see that the standard deviation for the CNN model was substantially tighter than the LSTM model, with it being 0.013 to 0.030, comparatively. This means there was more stability across the different seeds for the CNN, and the LSTM's accuracy varied more. The numbers found in this section will be the baseline used when comparing it to the input + output, and will allow us to see how much of an impact including the input has.

Input + Output:



It does not seem like there is any benefit to including both the input and the output for either the CNN model or the LSTM model. All the confusion matrices follow about the same pattern with no major differences. The CNN model is still pretty strong at identifying Claude-v1 responses, however got slightly worse at distinguishing between GPT-4 and Koala-13b. The LSTM model still had the same difficulties distinguishing between GPT-4 and Koala-13b, except it got substantially worse at distinguishing Claude-v1 from GPT-4. Compared to the output-only confusion table, the LSTM model misclassified another 17 Claude-v1 outputs as GPT-4. For the CNN model, the macro-f1 value decreased from 0.819 to 0.792, which is enough to show that it

doesn’t seem to have any benefit, even though it has a standard deviation of 0.027 that would just get it to equal the output-only value. For the LSTM model, the macro-f1 value dropped from 0.659 to 0.575, which is a much more substantial loss compared to the CNN decrease. It has a standard deviation of 0.047, so the output-only value would likely not be within a single standard deviation. One reason I would expect the LSTM to have decreased much more than the CNN model is that the input + output sequence the response now starts after a variable-length prompt and <SEP>, so its opener no longer sits at a consistent position, and the prompt uses up part of the max_len budget, which truncates more of the outputs ending. Both can hurt a sequential model more than the CNN, which uses max-pooling so it has an easier time finding patterns wherever they appear. And an overall reason why I don’t think either made any improvement overall is that the prompt introduces more noise and just makes it more difficult for the model. This can be seen since the input-only testing showed that the models basically performed at chance, so it doesn’t seem to contain much relevant information to benefit a particular class.

Lessons & Experience

Some good experience I got was with all the different machine learning Python libraries such as PyTorch and scikit-learn features. I had not yet had much experience, so it was good to gain more experience actually using these tools and being able to implement the features we are learning about in class. One lesson I learned that I had never thought about was the benefits of using a GPU to train the models rather than the CPU. We learned in class that the LSTM is much slower than the CNN, but after initially trying to set everything up in VS Code and running in a normal Python script using my CPU each epoch for the LSTM took like 3-4 minutes, and it was good to see how slow it can truly be in practice. This is also the first project I’ve had where I worked through a whole machine learning pipeline in this kind of way. I had to build the vocabulary, tokenize it, set up all the training aspects, and I think it was a great experience for me to go through the entire process and even go through all the debugging to make sure it worked as intended.

As far as the models go, I learned that having more input does not mean it’s better input. Adding the prompt to the response didn't help either model and hurt the LSTM. I also think overall I gained a better understanding of CNNs and LSTMs and the concepts behind them compared to if I just read the lecture notes. For example, I got a better idea of how the CNN's max-pooling finds a stylistic pattern wherever it appears in a response, while the LSTM's reading order and fixed sequence length made it more sensitive to where a response began and ended, which helped explain one possible reason why it dropped more when the prompt was added. I also thought it was interesting going through the fine-tuning process and running all the different parameters to figure out which would give the best results. Doing this also introduced me to learning about a scheduler, which lowers the learning rate during training when the validation loss stops improving. It was good to learn about stuff like this that isn’t always included in lecture notes and is something that you figure out by actually doing a project and researching different ways to make improvements.

I also learned a lot through the data collection process. I had never had to be so thorough in finding data or figuring out if certain data would work or not before because usually in a learning setting it is given to us. But it created a lot of learning opportunities to find it and build the particular dataset myself. The first dataset I found had plenty of models to choose from and I initially chose one where all the prompts would be the same for each model, and then each model would give its response. Then, as I worked through the project, I realized that if I did that the experiment I did for RQ2 would have no value because having identical prompts would not add any value to doing input-only testing. So then I had to go back and find another dataset that might work better. It was also entertaining writing a short Python script to format the data I got from Hugging Face into the type of format I wanted to use for this project, and was able to practice some techniques I haven't had to use in a while.

Overall, I thought this project was beneficial in increasing my knowledge and experience with lots of machine learning Python tools, and also making me think deeper about the architecture of the CNN and LSTM models and the ways to implement them. This experience was able to give me a foundation in the full machine learning workflow, from preparing the data to evaluating the models, that I can build on in future projects.